

**LAPORAN PRAKTIKUM SISTEM OPERASI
CPU SCHEDULING**



Dosen Pengampu:

Triawan Adi Cahyanto M.Kom

Disusun Oleh:

Yusa Salasa (2410651018)

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER**

2025

1. Eksperimen CPU Scheduling

Percobaan menggunakan data 1:

A. First Come First Served

```
yos@yos-QEMU-Virtual-Machine:~$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 3

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== GANTT CHART ===
|***
0 10 15 23
```

B. Shorted Job First

```
yos@yos-QEMU-Virtual-Machine:~$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 4

Proses P1
Arrival Time: 0
Burst Time: 5

Proses P2
Arrival Time: 1
Burst Time: 3

Proses P3
Arrival Time: 2
Burst Time: 8

Proses P4
Arrival Time: 3
Burst Time: 6

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0      5      5      5      0
P2      1      3      8      7      4
P3      2      8     16     14      6
P4      3      6     22     19     13

Average Turnaround Time: 11.25
Average Waiting Time: 5.75

=== GANTT CHART ===
|****
0 5 8 16 22
```

Percobaan menggunakan data 2 :

A. First Come First Served

```
yos@yos-QEMU-Virtual-Machine:~$ ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 3

Proses P1
Arrival Time: 0
Burst Time: 10
Proses P2
Arrival Time: 1
Burst Time: 5
Proses P3
Arrival Time: 2
Burst Time: 8

=== HASIL PENJADWALAN SJF ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== SEQUENCE ===
Proses dieksekusi dalam urutan: P1 -> P2 -> P3
```

B. Shorted Job First

```
yos@yos-QEMU-Virtual-Machine:~$ ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 4

Proses P1
Arrival Time: 0
Burst Time: 5
Proses P2
Arrival Time: 1
Burst Time: 3
Proses P3
Arrival Time: 2
Burst Time: 8
Proses P4
Arrival Time: 3
Burst Time: 6

=== HASIL PENJADWALAN SJF ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0      5      5      5      0
P2      1      3      8      7      4
P3      2      8     22     20     12
P4      3      6     14     11      5

Average Turnaround Time: 10.75
Average Waiting Time: 5.25

=== SEQUENCE ===
Proses dieksekusi dalam urutan: P1 -> P2 -> P4 -> P3
```

2. Analisis Eksperimen ketiga algoritma

Jawab pertanyaan berikut:

1. Algoritma mana yang memberikan Average Waiting Time terkecil?
2. Mengapa hasil algoritma berbeda-beda?
3. Kapan sebaiknya menggunakan algoritma FCFS?
4. Kapan sebaiknya menggunakan algoritma SJF?
5. Kapan sebaiknya menggunakan algoritma Round Robin?

Jawaban:

1. Algoritma SJF (Shortest Job First) menghasilkan rata-rata waktu tunggu paling rendah. Baik pada *Data Set 1* maupun *Data Set 2*, algoritma ini consistently memberikan *Average Waiting Time* yang lebih kecil dibandingkan dengan FCFS maupun Round Robin.
2. Perbedaan hasil di antara ketiga algoritma disebabkan oleh kebijakan penjadwalan yang berbeda-beda dalam menentukan urutan eksekusi proses. Setiap algoritma memiliki cara sendiri dalam memprioritaskan proses.
3. FCFS (First Come First Served) cocok digunakan ketika:
 - Sistem menangani banyak proses yang datang secara berurutan, misalnya pada *batch processing* atau *job queue*.
 - Variasi waktu eksekusi antar proses tidak menjadi masalah besar.
 - Tujuan utamanya adalah kesederhanaan dan keadilan berdasarkan urutan kedatangan proses.
4. SJF (Shortest Job First) lebih tepat digunakan ketika:
 - Waktu eksekusi (*burst time*) tiap proses dapat diketahui atau diperkirakan sebelumnya.
 - Fokus sistem adalah mencapai efisiensi maksimal dengan meminimalkan *waiting time* dan *turnaround time*.
 - Proses-proses dengan durasi pendek lebih sering muncul dibanding proses panjang.
5. Round Robin (RR) paling sesuai diterapkan pada sistem interaktif atau multitasking, di mana setiap proses perlu mendapat jatah CPU secara bergiliran.
 - Tujuan utamanya adalah memberikan *fairness* bagi semua proses dan menjaga waktu respons agar tetap cepat terhadap pengguna.

3. Modifikasi Program

A. Tugas Tambahkan fitur berikut pada program FCFS:

1. Tampilkan Gantt Chart yang lebih detail
2. Simpan hasil ke file text
3. Baca input dari file

B. Langkah Langkah

- Tambahkan 3 method pada program fcfs yang kita sudah buat

```
GNU nano 8.4 fcfs.c *
#include <stdio.h>
#include <stdlib.h>

struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
};

int readInputFromFile(const char *filename, struct Process proc[]) {
    FILE *f = fopen(filename, "r");
    if (f) {
        printf("Gagal membuka file input!\n");
        return 0;
    }
    int n;
    fscanf(f, "%d", &n);
    for (int i = 0; i < n; i++) {
        fscanf(f, "%d %d %d", &proc[i].pid, &proc[i].arrival_time, &proc[i].burst_time);
    }
    fclose(f);
    return n;
}

void saveResultToFile(const char *filename, struct Process proc[], int n, float avg_tat, float avg_wt) {
    FILE *f = fopen(filename, "w");
    if (f) {
        printf("Gagal membuat file hasil!\n");
        return;
    }
    fprintf(f, "==== HASIL PENJAJALAN FCFS ====\n");
    fprintf(f, "PID\tAT\tBT\tCT\tTAT\tWT\n");
    fprintf(f, "---[t--[t--[t--[t--[t--[t--\n");

    for (int i = 0; i < n; i++) {
        fprintf(f, "%d\t%d\t%d\t%d\t%d\t%d\n",
            proc[i].pid,
            proc[i].arrival_time,
            proc[i].burst_time,
            proc[i].completion_time,
            proc[i].turnaround_time,
            proc[i].waiting_time);
    }

    fprintf(f, "\nAverage Turnaround Time: %.2f\n", avg_tat);
    fprintf(f, "Average Waiting Time: %.2f\n", avg_wt);
    fprintf(f, "\n==== GANTT CHART ====\n");

    for (int i = 0; i < n; i++) {
        fprintf(f, "P%d (%d-%d) ", proc[i].pid,
            (i == 0 ? proc[i].completion_time - proc[i].burst_time : proc[i - 1].completion_time),
            proc[i].completion_time);
    }
    fprintf(f, "\n");
    fclose(f);
    printf("Hasil juga disimpan ke file 'hs'\n", filename);
}

int main() {
    struct Process proc[20];
    int n;

    printf("==== SIMULASI ALGORITMA FCFS (MODIFIKASI) ====\n");

    n = readInputFromFile("input_fcfs.txt", proc);
    if (n == 0) return 0;

    int current_time = 0;
    for (int i = 0; i < n; i++) {
        if (current_time < proc[i].arrival_time)
            current_time = proc[i].arrival_time;
        proc[i].completion_time = current_time + proc[i].burst_time;
        proc[i].turnaround_time = proc[i].completion_time - proc[i].arrival_time;
        proc[i].waiting_time = proc[i].turnaround_time - proc[i].burst_time;
        current_time = proc[i].completion_time;
    }

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    printf("---[t--[t--[t--[t--[t--[t--\n");

    float total_tat = 0, total_wt = 0;
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            proc[i].pid,
            proc[i].arrival_time,
            proc[i].burst_time,
            proc[i].completion_time,
            proc[i].turnaround_time,
            proc[i].waiting_time);
        total_tat += proc[i].turnaround_time;
        total_wt += proc[i].waiting_time;
    }

    float avg_tat = total_tat / n;
    float avg_wt = total_wt / n;

    printf("\nAverage Turnaround Time: %.2f\n", avg_tat);
    printf("Average Waiting Time: %.2f\n", avg_wt);

    printf("==== GANTT CHART ====\n");
    for (int i = 0; i < n; i++) {
        printf("P%d (%d-%d) ", proc[i].pid,
            (i == 0 ? proc[i].completion_time - proc[i].burst_time : proc[i - 1].completion_time),
            proc[i].completion_time);
    }
    printf("\n");

    saveResultToFile("hasil_fcfs.txt", proc, n, avg_tat, avg_wt);
    return 0;
}
```

- Lalu compile, setelah di compile masukan buat file incex.txt Dan masukan text seperti format di bawah ini

```
GNU nano 8.4
3
1 0 10
2 1 5
3 2 8
```

- Jika di run maka akan keluar output seperti di bawah ini :

```
yos@yos-QEMU-Virtual-Machine:~$ nano fcfs_modifikasi.c
yos@yos-QEMU-Virtual-Machine:~$ gcc fcfs_modifikasi.c -o fcfs
yos@yos-QEMU-Virtual-Machine:~$ ./fcfs
=== SIMULASI ALGORITMA FCFS (MODIFIKASI) ===

PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        0      10      10      10        0
P2        1       5      15      14         9
P3        2       8      23      21        13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== GANTT CHART ===
| P1 (0-10) | P2 (10-15) | P3 (15-23) |
Hasil juga disimpan ke file 'hasil_fcfs.txt'
```

4. Eksperimen dengan data berbeda untuk perbandingan algoritma

A. Proses dengan burst time bervariasi

```
Masukkan jumlah proses: 4
Masukkan time quantum: 3

Proses P1
Arrival Time: 0
Burst Time: 24
Proses P2
Arrival Time: 1
Burst Time: 3
Proses P3
Arrival Time: 2
Burst Time: 3
Proses P4
Arrival Time: 3
Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1
-> Sisa waktu: 21
Waktu 3: Menjalankan P1
-> Sisa waktu: 18
Waktu 6: Menjalankan P2
Waktu 9: Menjalankan P3
Waktu 12: Menjalankan P4
-> Sisa waktu: 3
Waktu 15: Menjalankan P1
-> Sisa waktu: 15
Waktu 18: Menjalankan P1
-> Sisa waktu: 12
Waktu 21: Menjalankan P1
-> Sisa waktu: 9
Waktu 24: Menjalankan P4
Waktu 27: Menjalankan P4

=== HASIL PENJADWALAN ROUND ROBIN ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        0      24       0       0         0
P2        1       3       9       8         5
P3        2       3      12      10         7
P4        3       6      27      24        18

Average Turnaround Time: 10.50
Average Waiting Time: 7.50
yos@yos-QEMU-Virtual-Machine:~$
```

B. Proses datang bersamaan

```
yos@yos-QEMU-Virtual-Machine:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 5
Masukkan time quantum: 4

Proses P1
Arrival Time: 0
Burst Time: 10
Proses P2
Arrival Time: 0
Burst Time: 5
Proses P3
Arrival Time: 0
Burst Time: 8
Proses P4
Arrival Time: 0
Burst Time: 12
Proses P5
Arrival Time: 0
Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1
-> Sisa waktu: 6
Waktu 4: Menjalankan P2
-> Sisa waktu: 1
Waktu 8: Menjalankan P3
-> Sisa waktu: 4
Waktu 12: Menjalankan P4
-> Sisa waktu: 8
Waktu 16: Menjalankan P5
-> Sisa waktu: 2
Waktu 20: Menjalankan P1
-> Sisa waktu: 2
Waktu 24: Menjalankan P1
Waktu 26: Menjalankan P2
Waktu 27: Menjalankan P2
Waktu 27: Menjalankan P3
Waktu 31: Menjalankan P3

=== HASIL PENJADWALAN ROUND ROBIN ===
PID  AT   BT   CT   TAT   WT
---  --   --   --   ---   --
P1    0    10   26    26    16
P2    0     5   27    27    22
P3    0     8   31    31    23
P4    0    12  62691  1745097776  0
P5    0     6     0   62691  1745228680

Average Turnaround Time: 349032128.00
Average Waiting Time: 349044160.00
yos@yos-QEMU-Virtual-Machine:~$
```

C. Proses datang dengan interval

```
yos@yos-QEMU-Virtual-Machine:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 5
Masukkan time quantum: 2

Proses P1
Arrival Time: 0
Burst Time: 8
Proses P2
Arrival Time: 2
Burst Time: 4
Proses P3
Arrival Time: 4
Burst Time: 9
Proses P4
Arrival Time: 6
Burst Time: 5
Proses P5
Arrival Time: 8
Burst Time: 3

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1
-> Sisa waktu: 6
Waktu 2: Menjalankan P1
-> Sisa waktu: 4
Waktu 4: Menjalankan P2
-> Sisa waktu: 2
Waktu 6: Menjalankan P1
-> Sisa waktu: 2
Waktu 8: Menjalankan P1
Waktu 10: Menjalankan P3
-> Sisa waktu: 7
Waktu 12: Menjalankan P1
Waktu 12: Menjalankan P2
Waktu 14: Menjalankan P4
-> Sisa waktu: 3
Waktu 16: Menjalankan P1
Waktu 16: Menjalankan P1

=== HASIL PENJADWALAN ROUND ROBIN ===
PID  AT   BT   CT   TAT   WT
---  --   --   --   ---   --
P1    0     8   16    16     8
P2    2     4   16    14    10
P3    4     9     0   65535 -609162608
P4    6     5  62287 -886041552     0
P5    8     3     0   62287 -885898168

Average Turnaround Time: -177182752.00
Average Waiting Time: -299012160.00
yos@yos-QEMU-Virtual-Machine:~$
```

5. Analisis Eksperimen Perbandingan Algoritma

1. Analisis Performa

FCFS (First Come First Served) menunjukkan performa yang cukup baik (dengan *Average Waiting Time* atau AWT mendekati algoritma lain) pada kondisi tertentu, yaitu ketika:

- Burst Time (BT) setiap proses hampir sama besar. Dalam situasi seperti ini, urutan eksekusi proses tidak banyak memengaruhi total waktu tunggu.
- Proses berdurasi pendek datang terlebih dahulu. Jika proses kecil tiba lebih dulu, FCFS dapat menyelesaikannya dengan cepat dan menghasilkan AWT yang relatif kecil.
- Tidak terjadi Convoy Effect. FCFS akan bekerja optimal bila tidak ada proses besar yang datang di awal antrian. Jika ada proses dengan BT besar (misalnya P1: BT = 24 pada percobaan pertama), proses-proses kecil akan menunggu lama — inilah yang disebut *Convoy Effect*.

Mengapa SJF (Shortest Job First) hampir selalu unggul dalam AWT?

SJF dikenal sebagai algoritma yang paling efisien dalam meminimalkan waktu tunggu rata-rata karena pendekatannya yang berfokus pada proses dengan durasi terpendek terlebih dahulu.

- Dengan menjalankan proses kecil lebih awal, sistem secara tidak langsung mengurangi waktu tunggu bagi sebagian besar proses berikutnya.
- Strategi ini membuat *waiting time* total menjadi yang paling kecil di antara algoritma lainnya, menjadikan SJF optimal dalam konteks ini.

2. Context Switching

Algoritma dengan jumlah context switch terbanyak adalah Round Robin (RR).

Hal ini terjadi karena setiap kali jatah waktu (*time quantum*) habis, CPU harus berpindah ke proses berikutnya di antrian. Proses pergantian ini menimbulkan *context switch* yang cukup sering.

Pengaruh besar-kecilnya quantum terhadap jumlah context switch pada RR:

- Quantum kecil: jumlah *context switch* meningkat tajam. Jika quantum sangat kecil (misalnya 1), CPU harus berpindah hampir setiap satuan waktu, menyebabkan *overhead* tinggi dan efisiensi menurun.
- Quantum besar: jumlah *context switch* berkurang. Dengan quantum yang besar, RR mulai berperilaku menyerupai FCFS, karena satu proses bisa menyelesaikan eksekusinya sebelum jatah waktunya habis.

Eksperimen dengan variasi quantum (2, 4, 8, 16):

- Pada quantum kecil (2 atau 4), sistem memberikan respon yang cepat terhadap setiap proses (response time rendah), namun jumlah *context switch* meningkat tajam sehingga AWT menjadi lebih tinggi.
- Pada quantum lebih besar (8 atau 16), pergantian proses berkurang dan efisiensi meningkat, tetapi waktu respons terhadap proses baru menjadi lebih lambat.